Artificial Intelligence Lab

Dibyasundar Das

24 September 2026

LAB 08



- Write a program that generates a random Wumpus World and runs an agent that explores it without knowing the map. The agent reports **POSSIBLE** with the actions that reach and grab the gold, or **NOT POSSIBLE**. It must never enter a cell unless it can prove the cell is safe. The details of the world is as follows :
  - The grid is N×N (default 4). Cells are (x, y), with (1,1) at the bottom-left. The agent starts at (1,1) and knows N.
  - There is exactly **one Wumpus**, never at (1,1). It **may share a cell with a pit**.
  - Each cell except (1,1) has a **pit** with probability p (default 0.2).
  - There is exactly **one gold**, in a cell with no pit and no Wumpus.
- **Percepts** (the agent senses only its current cell)
  - **Stench:** the Wumpus is in a neighbouring cell (up, down, left or right). A dead Wumpus still smells.
  - **Breeze:** at least one neighbouring cell has a pit.
  - **Glitter:** the gold is in this cell.
  - **Scream:** the Wumpus has just been killed.
- **Actions**
  - `UP`, `DOWN`, `LEFT`, `RIGHT` move one cell.
  - `SHOOT_<dir>` fires the **single arrow**, which flies in a straight line to the wall.

    - `GRAB` picks up the gold.
- **What the agent tracks by using rule based system**
    - `Visited`.
    - `PitFree` : visited cells, plus every neighbour of a visited cell with no Breeze.
    - `WumpusCandidates` :
        - It starts as every cell except (1,1).
        - Each Stench cell keeps only the candidates next to it.
        - Each visited cell with no Stench removes itself and its neighbours.
        - A shot with no Scream removes the arrow's line.
        - A Scream sets `WumpusDead`.

> **Note**
>
> A cell is **safe** iff it is in `PitFree` **and** (`WumpusDead` **or** the cell is not a candidate). Killing the Wumpus does **not** make its cell pit-free.

- **Agent loop**
    1. If there is Glitter, `GRAB` and stop.
    2. Otherwise, use BFS through visited cells to reach the nearest unvisited safe cell.
    3. If there is no safe cell and the arrow is unused, shoot along a row or column that contains a candidate.
    4. If there is still no safe cell, report **NOT POSSIBLE**.
- **Input and output**
    - **Input:** either `N p seed`, or a world file with one object per line (`N 4`, `W x y`, `P x y`, `G x y`).
    - **Output:**
        - A trace with the action, percept and new safe cells for each step.
        - The verdict.
        - If POSSIBLE, the **shortest plan** through cells known to be safe. Any valid shortest plan is accepted.
        - The true map, printed only at the end.

Test Cases

`Legend: A = agent start (1,1), W = Wumpus, P = pit, G = gold, W+P = Wumpus and pit in the same cell, · = empty`

Test 1:

```
| y \ x | 1 | 2 | 3 | 4 |
|:-----:|:-:|:-:|:-:|:-:|
|   4   | · | · | · | P |
```

```
|   3   | W | G | P | · |
|   2   | · | · | · | · |
|   1   | A | · | P | · |

Output: RIGHT, UP, UP, GRAB` (path (1,1) → (2,1) → (2,2) → (2,3))
```

Test 2:

```
| y \ x |  1  |  2  |  3  |  4  |
| :---: | :-: | :-: | :-: | :-: |
|   4   |  ·  |  ·  |  ·  |  ·  |
|   3   |  ·  |  ·  |  P  |  ·  |
|   2   |  ·  |  ·  |  G  |  ·  |
|   1   |  A  |  W  |  ·  |  P  |

Output: SHOOT_RIGHT (Scream), RIGHT, RIGHT, UP, GRAB (path (1,1) → (2,1) →
(3,1) → (3,2))
```

Test 3:

```


| y \ x |  1  |  2  |  3  |  4  |
|:-----:|:---:|:---:|:---:|:---:|
|   4   |  ·  |  ·  |  ·  |  G  |
|   3   |  ·  |  ·  |  ·  |  ·  |
|   2   |  ·  |  ·  |  ·  |  ·  |
|   1   |  A  | W+P |  ·  |  ·  |

Output: NOT POSSIBLE
```